RExercise#8_Porras

Renz Benedict Porras

2026-11-06

Part A — Creating Data Frames

Question #1:

```
# a. Create a vector

scores <- c(Ana = 78, Ben = 92, Cid = 85, Dex = 92, Eli = 67, Fay = 88, Gio = 74)
scores
```

```
## Ana Ben Cid Dex Eli Fay Gio
##  78  92  85  92  67  88  74
```

```
# b. Sort the scores in ascending order and descending order. What is the output?

# Ascending order:
sort(scores)
```

```
## Eli Gio Ana Cid Fay Ben Dex
##  67  74  78  85  88  92  92
```

```
# Descending order:
sort(scores, decreasing = TRUE)
```

```
## Ben Dex Fay Cid Ana Gio Eli
##  92  92  88  85  78  74  67
```

```
# c. Print your sorted result from sort-ascending. Check whether the # names ("Ana", "Ben", ...) stayed

sort_asc <- sort(scores)
sort_asc
```

```
## Eli Gio Ana Cid Fay Ben Dex
##  67  74  78  85  88  92  92
```

```
# Explanation:
# Yes, the names remain attached to their correct numeric values because named vectors in R preserve el
```

```
# d. Create another vectors:

subjects <- c("Science", "Math", "English", "Filipino", "PE")
incomplete <- c(85, NA, 90, 70, NA, 95)

subjects
```

```
## [1] "Science"  "Math"     "English"  "Filipino" "PE"
```

```
incomplete
```

```
## [1] 85 NA 90 70 NA 95
```

```
# e. Sort subjects alphabetically using sort()

sort(subjects)
```

```
## [1] "English"  "Filipino" "Math"     "PE"       "Science"
```

```
# f. Sort incomplete using sort(). Observe what happens to the NA

# values by default.
# TODO: Now sort it again using na.last = TRUE so the NAs appear at the end

# Default sort:
sort(incomplete)
```

```
## [1] 70 85 90 95
```

```
# Sorted with NAs at the end:
sort(incomplete, na.last = TRUE)
```

```
## [1] 70 85 90 95 NA NA
```

Question #2:

```
# When you sorted scores, did the names stay correctly attached to their original values?
# Explain why this matters: if you were sorting exam scores to find the top performer, what would go wr

# Explanation:
# Yes, the names stayed correctly attached to their original values.
# This matters because if names did not move alongside their respective scores, student identity would
```

Question #3:

```
# By default, sort() silently removes NA values instead of showing them or throwing an error.
# Why might this default behavior be dangerous if you were reporting "the lowest score" in a dataset wi

# Explanation:
# Silently dropping NAs can hide missing data issues (e.g., unentered scores or absent students). If NA
```

Question #4:

```
# Use order(scores) and print the result. Compare this output to the sort(scores) output. What is the d

order(scores)
```

```
## [1] 5 7 1 3 6 2 4
```

```
sort(scores)
```

```
## Eli Gio Ana Cid Fay Ben Dex
##  67  74  78  85  88  92  92
```

```
# Explanation:
# order(scores) returns the vector of position indices that would sort the elements (e.g., 5, 7, 1, 3, 
```

```
# a. Use the result of order(scores) to index scores itself: # Compare this to sort(scores) - they shou

# Do they have the same values? Explain briefly.

scores[order(scores)]
```

```
## Eli Gio Ana Cid Fay Ben Dex
##  67  74  78  85  88  92  92
```

```
# Explanation:
# Yes, they produce the exact same values. Indexing scores by its ordered position vector rearranges th
```

Question #5:

```
# Create a dataframe from these vectors:
# name = c("Ana", "Ben", "Cid", "Dex", "Eli", "Fay", "Gio")
# score = c(78, 92, 85, 92, 67, 88, 74)
# section = c("A", "B", "A", "B", "A", "B", "A")
# What is the output?

students <- data.frame(
  name = c("Ana", "Ben", "Cid", "Dex", "Eli", "Fay", "Gio"),
  score = c(78, 92, 85, 92, 67, 88, 74),
  section = c("A", "B", "A", "B", "A", "B", "A")
)

students
```

```
##   name score section
## 1  Ana    78       A
## 2  Ben    92       B
## 3  Cid    85       A
## 4  Dex    92       B
## 5  Eli    67       A
## 6  Fay    88       B
## 7  Gio    74       A
```

```
# a. Reorder the ENTIRE students data frame by score using students[order(students$score), ]

# What is the output?

students[order(students$score), ]
```

```
##   name score section
## 5  Eli    67       A
## 7  Gio    74       A
## 1  Ana    78       A
## 3  Cid    85       A
## 6  Fay    88       B
## 2  Ben    92       B
## 4  Dex    92       B
```

```r
# Output:
# Displays the full students data frame with all rows reordered from lowest score (Eli, 67) to highest
```

```r
# b. Reorder students by score in DESCENDING order.

students[order(students$score, decreasing = TRUE), ]
```

```
##   name score section
## 2  Ben    92       B
## 4  Dex    92       B
## 6  Fay    88       B
## 3  Cid    85       A
## 1  Ana    78       A
## 7  Gio    74       A
## 5  Eli    67       A
```

Question #6:

```r
# sort(scores) and scores[order(scores)] produce the same final values, but they work completely differ
# In your own words, explain what order(scores) actually returns (look at the raw numbers it gives you)

# Explanation:
# order(scores) returns an integer vector of positional indices showing the order in which elements app
# On its own, this output is useless because it only provides position numbers rather than the actual d
```

Question #7:

```r
# Explain why sort() alone could NOT have been used to reorder the whole students data frame by score (
# Why is order() the correct tool whenever you need to keep multiple related columns lined up with each

# Explanation:
# 1. Why sort() fails on data frames: sort() only operates on single 1D vectors. A data frame is a 2D t
# 2. Why order() is the correct tool: order(students$score) generates the exact sequence of row index n
```

Question #8:

```r
# TODO: Use rank(scores) and print the result.
# Print scores, then rank(scores), side by side (e.g., using
# cbind(scores, rank(scores))) so you can see which score got which rank

rank(scores)
```

```
## Ana Ben Cid Dex Eli Fay Gio
## 3.0 6.5 4.0 6.5 1.0 5.0 2.0
```

```r
cbind(scores, rank = rank(scores))
```

```
##     scores rank
## Ana     78  3.0
## Ben     92  6.5
## Cid     85  4.0
## Dex     92  6.5
```

```
## Eli     67  1.0
## Fay     88  5.0
## Gio     74  2.0
```

```
# a. Ranking the output for Ben and Dex
# TODO: Look at the rank() output for Ben and Dex.
# What rank did each of them receive?

# Answer:
# Both Ben and Dex received a rank of 6.5.
```

```
# b. Ben and Dex both scored 92, the highest score in the class, but under the default rank() they did

# Explanation:
# - Value received: 6.5
# - Reason: By default, rank() uses ties.method = "average". Ben and Dex occupy the 6th and 7th positio
```

Question #8:

```
# a. Create a dataframe.
# sales <- data.frame( employee = c("Rey", "Sam", "Tin", "Uma", "Val"), units_sold = c(120, 145, 145, 9
# Add a column sales$rank that ranks employees by units_sold, with the TOP seller receiving rank 1.

sales <- data.frame(
  employee = c("Rey", "Sam", "Tin", "Uma", "Val"),
  units_sold = c(120, 145, 145, 98, 130)
)

sales$rank <- rank(-sales$units_sold)
sales
```

```
##   employee units_sold rank
## 1      Rey        120  4.0
## 2      Sam        145  1.5
## 3      Tin        145  1.5
## 4      Uma         98  5.0
## 5      Val        130  3.0
```

```
# b. Reorder the entire sales data frame using order() so it is displayed from HIGHEST units_sold to LO

sales[order(sales$units_sold, decreasing = TRUE), ]
```

```
##   employee units_sold rank
## 2      Sam        145  1.5
## 3      Tin        145  1.5
## 5      Val        130  3.0
## 1      Rey        120  4.0
## 4      Uma         98  5.0
```

Question #9:

```
# In this challenge, you used rank() to determine standing and order() to determine display order.
# Could you have used order() alone to solve the entire problem (both awarding ranks AND reordering the
# Explain what order() is missing that rank() provides, using your own words rather than repeating the
```

```
# Explanation:
# No, order() alone could not solve the entire problem.
# While order() provides the row indices needed to sort the table physically, it cannot handle ties pro

# What order() is missing:
# 1. Proper Tie Handling: order() forces a sequential breakdown for identical values (assigning one tie
# 2. Direct Value Mapping: order() returns row index positions for sorting rather than calculating quan
```